DSA5208: Scalable Distributed Computing for Data Science

Lecture 1: Physical and Logical Times

Zhenning Cai

Department of Mathematics, National University of Singapore

AY2026/2027, Semester 1

1 Models of distributed systems

2 Physical and logical times

3 Summary

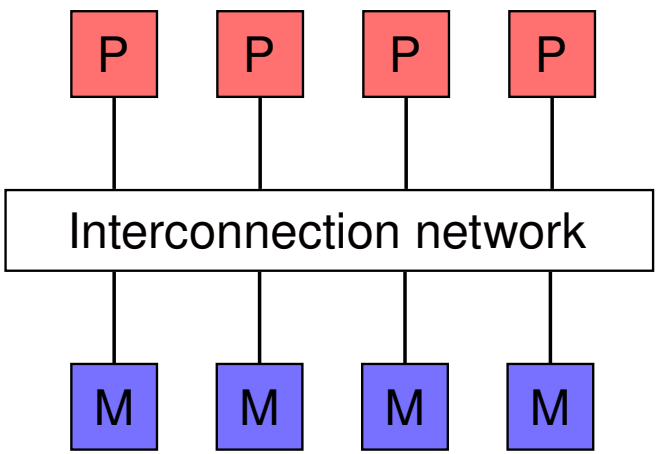
Multiprocessor and multicomputer systems

A distributed system is a collection of independent entities that cooperate to solve a problem.

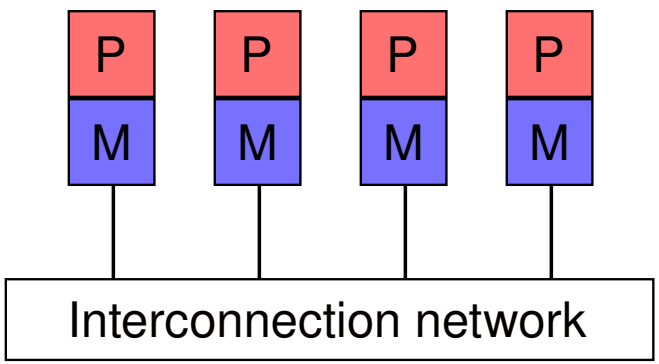
Multiprocessor and multicomputer systems

A distributed system is a collection of independent entities that cooperate to solve a problem.

- **A multiprocessor system:**



- **A multicomputer system:**

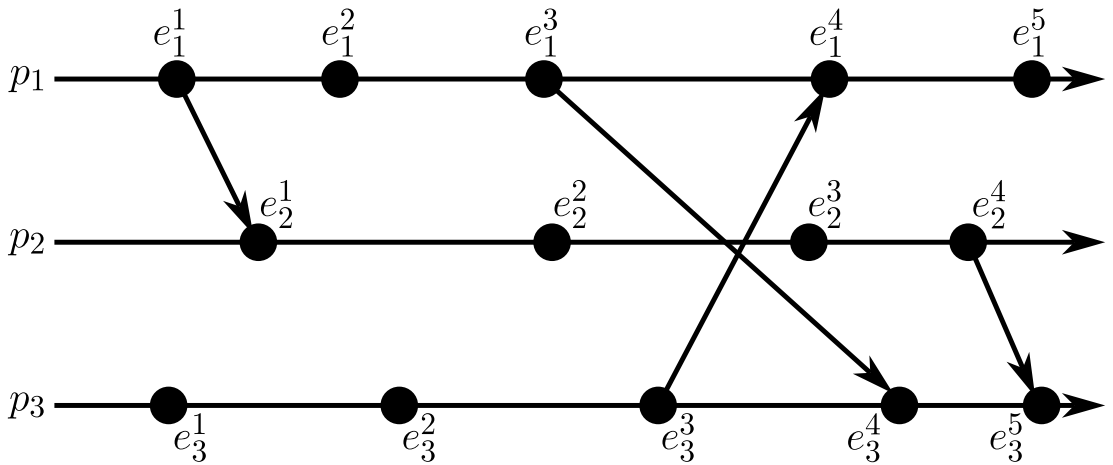


P = Processor M = Memory

A distributed diagram

Assumptions:

- The processes do not share a common global memory.
- The processes communicate solely by passing messages.



Notations:

- Processes: $p_1, p_2, \cdots$
- Events on process i: $h_i = \{e_i^1, e_i^2, \cdots\}$
- Set of all events: $H = \cup_i h_i$

Causal precedence relations

For $e_i^x, e_j^y \in H$,

$$e_i^x \to e_j^y \iff \begin{cases} i = j \text{ and } x < y, \\ \text{or} \\ e_i^x \text{ sends a message } m \text{ and } e_j^y \text{ receives } m, \\ \text{or} \\ \exists e_k^z \in H \text{ s.t. } e_i^x \to e_k^z \text{ and } e_k^z \to e_j^y. \end{cases}$$

The causal precedence relation "$\to$" is Lamport's "happens before" relation.

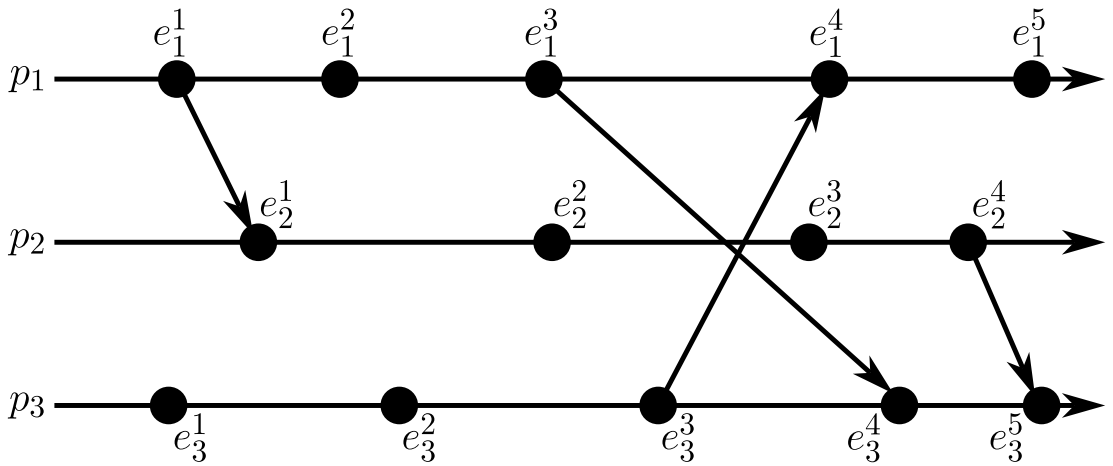
- "$e_i^x \to e_j^y$" is pronounced "e_i^x happens before e_j^y".
- "$\to$" is a partial order on H.

Causal precedence relations

Definitions:

- "$e_i^x \not\rightarrow e_j^y$" if "$e_i^x \rightarrow e_j^y$" does not hold.
- "$e_i^x \parallel e_j^y$" if "$e_i^x \not\rightarrow e_j^y$" and "$e_j^y \not\rightarrow e_i^x$".

Example:

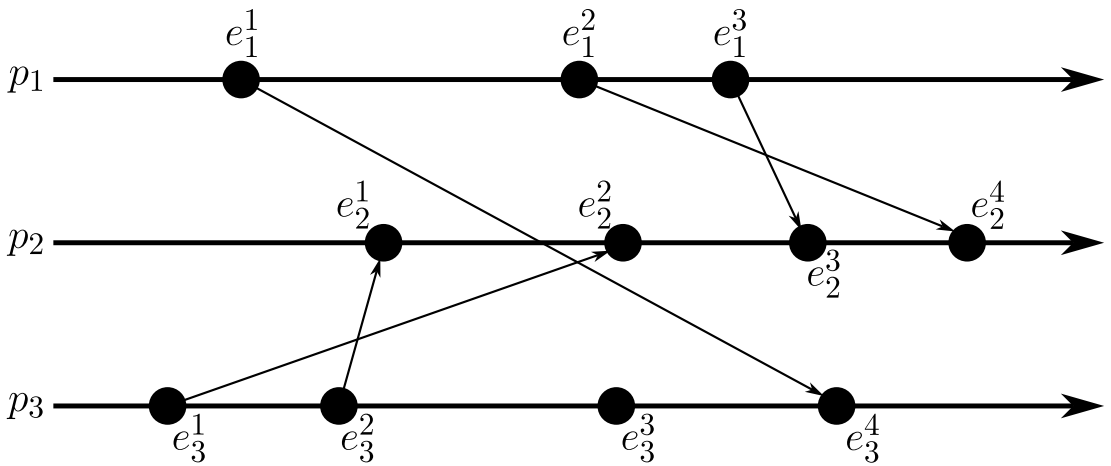


$$\boxed{e_1^1 \rightarrow e_3^5, \quad e_1^4 \not\rightarrow e_3^2, \quad e_1^3 \parallel e_3^3}$$

Note: $e_i^x \parallel e_j^y$ and $e_j^y \parallel e_k^z \quad \Longrightarrow \quad e_i^x \parallel e_k^z$.

Models of communication networks

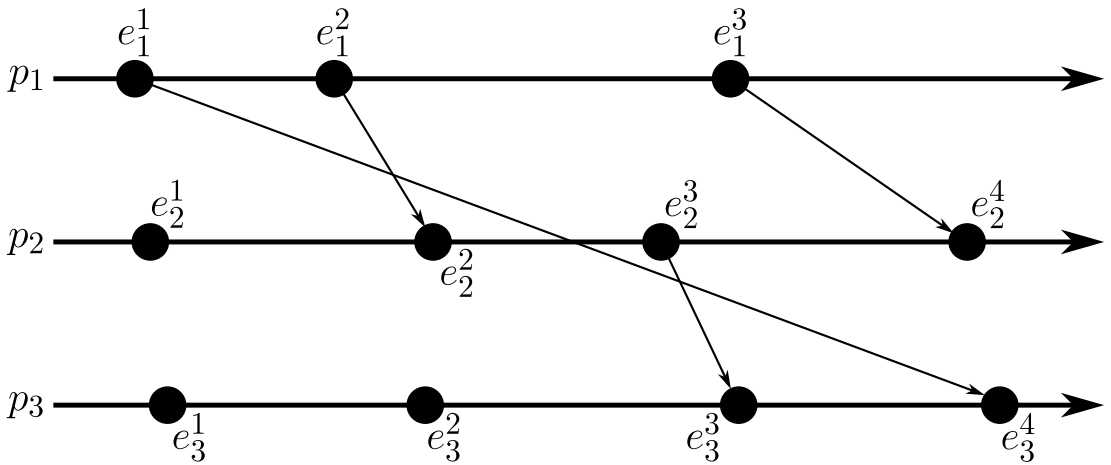
- **Non-FIFO model:** Messages in channels are received in a random order.



$$e_1^2 \rightarrow e_1^3 \text{ but } e_2^3 \rightarrow e_2^4$$
$$e_3^1 \rightarrow e_3^2 \text{ but } e_2^1 \rightarrow e_2^2$$

Models of communication networks

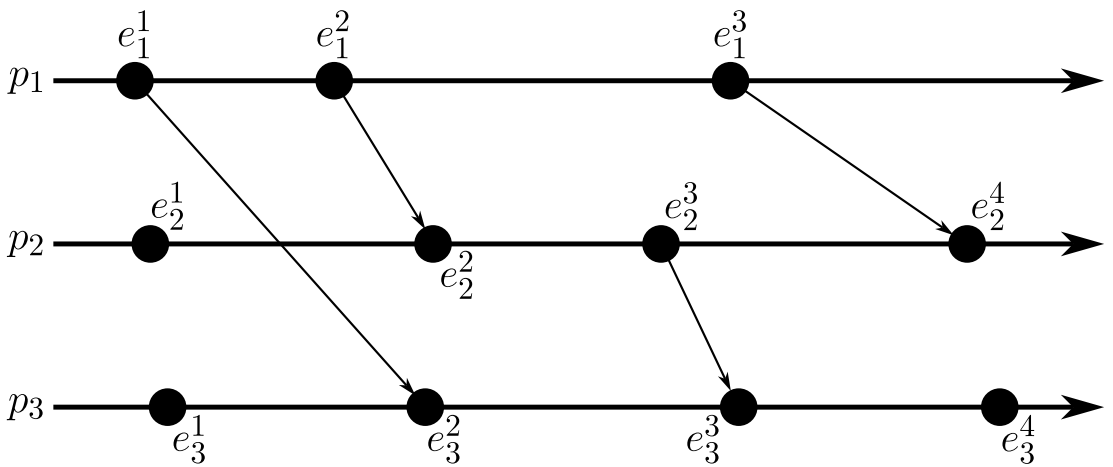
- **FIFO model:** Each channel acts as a first-in first-out message queue.



$$e_1^2 \rightarrow e_1^3 \text{ and } e_2^2 \rightarrow e_2^4$$
$$e_1^1 \rightarrow e_2^3 \text{ but } e_3^3 \rightarrow e_3^4$$

Models of communication networks

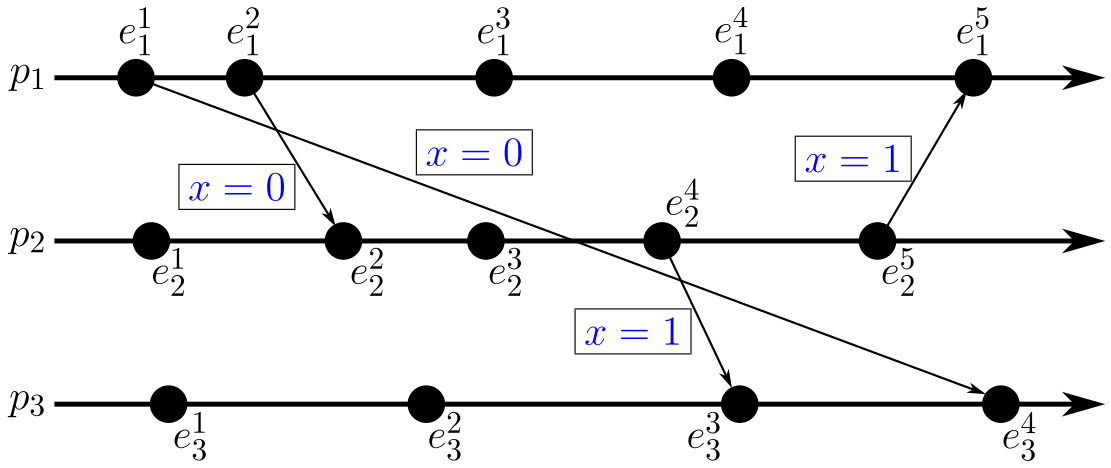
- **CO (Causal Ordering) model:** For any two messages m_1 and m_2 received by the same process p_k in events $e_k^{z_1}$ and $e_k^{z_2}$, respectively, if the event e_i^x sending m_1 and the event e_j^y sending m_2 satisfy $e_i^x \rightarrow e_j^y$, then $e_k^{z_1} \rightarrow e_k^{z_2}$.



$$e_1^2 \rightarrow e_1^3 \text{ and } e_2^2 \rightarrow e_2^4$$
$$e_1^1 \rightarrow e_2^3 \text{ and } e_3^2 \rightarrow e_3^3$$

Causal ordering

Without causal ordering:



- Causal ordering provides a built-in synchronization that usually guarantees consistency between processes.
- The implementation of causal ordering requires the buffer of the receiver to deliver messages in the correct order.
- A timestamp needs to be attached to each message.

1 Models of distributed systems

2 Physical and logical times

3 Summary

1 Models of distributed systems

2 Physical and logical times
- Physical clocks
- Logical times

3 Summary

Physical clock

Clocks that adhere to the physical time are called physical clocks.

Coordinated Universal Time (UTC):

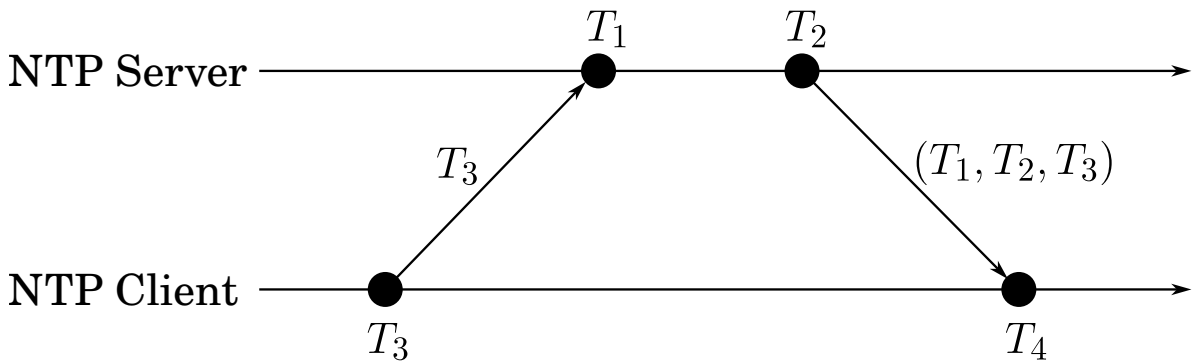
UTC = TAI (International Atomic Time) + (leap second correction)

TAI: 1s = 9,192,631,770 periods of caesium-133's resonant frequency

Unix time stamp: the number of seconds that have elapsed since 00:00:00 UTC on 1 January 1970, without adjustments made because of leap seconds.

- There is no global clock in a distributed system.
- Each process has its own internal clock.
- Clock drift causes the clock error to increase gradually.
- Clock synchronization is needed to generate reliable timestamps.

Network Time Protocol (NTP)



Round trip delay:

$$\delta = (T_4 - T_3) - (T_2 - T_1).$$

Estimated server time when the client receives the message:

$$T_2 + \frac{\delta}{2}.$$

1 Models of distributed systems

2 Physical and logical times
- Physical clocks
- Logical times

3 Summary

Logical clocks

Time domain: A partially ordered set $(T, <)$

Logical clock:

$$C : H \to T$$

Clock consistency:

$$e_i \to e_j \quad \Longrightarrow \quad C(e_i) < C(e_j)$$

Strong consistency:

$$e_i \to e_j \quad \Longleftrightarrow \quad C(e_i) < C(e_j)$$

Lamport's scalar time

Notation: The local logical time of process p_i is denoted by C_i.

Definition: The update of Lamport's scalar time ($T = \mathbb{N}$) follows two rules:

1. Before a send or internal event,

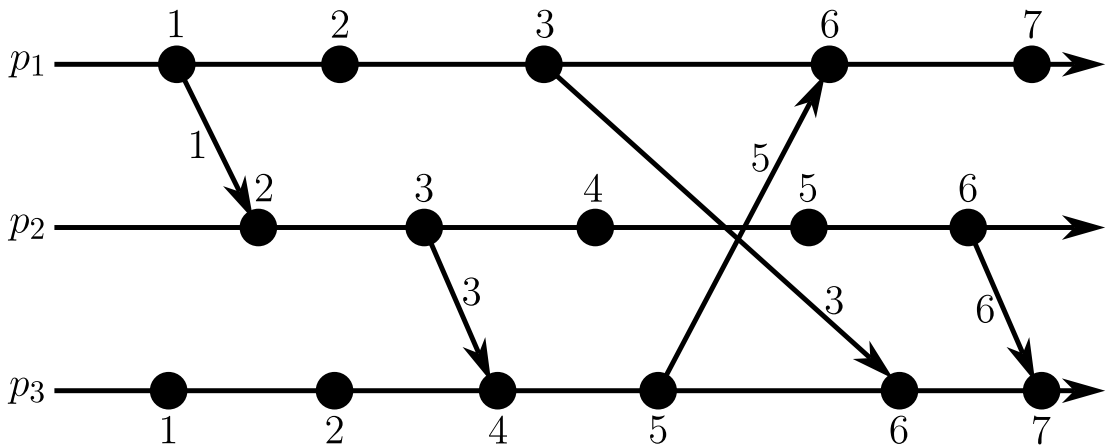
$$C_i := C_i + d, \qquad d > 0,$$

where d is often set to be 1, but it can also be application-dependent. The message to be sent piggybacks the clock value C_i after C_i is updated.

2. When p_i receives a message with timestamp C_{msg}, execute the following actions:
   1. $C_i := \max(C_i, C_{msg})$;
   2. Execute 1;
   3. Deliver the message.

Example

Assume $d \equiv 1$. All clocks begin with 0.



Properties

- **Consistency:** for two events e_i and e_j,

$$e_i \to e_j \implies C(e_i) < C(e_j),$$
$$C(e_i) = C(e_j) \implies e_i \parallel e_j.$$

- **Total ordering:**

$$e_i^x \prec e_j^y \iff C(e_i^x) < C(e_j^y) \text{ or } (C(e_i^x) = C(e_j^y) \text{ and } i < j).$$

  Total ordering is useful in the implementation of distributed systems to solve problems such as mutual exclusion[1].

- **Event counting:** If $d \equiv 1$, for any event e, $C(e) - 1$ events have been produced sequentially before e.
- **No strong consistency:** For two events e_1 and e_2, it is possible that $e_1 \parallel e_2$ but $C(e_1) < C(e_2)$.

[1] See "L. Lamport. 1978. Time, clocks, and the ordering of events in a distributed system. Commun. ACM 21, 7 (July 1978), 558–565."

Vector time

Notation: The local logical time of process p_i is denoted by $vt_i \in \mathbb{N}^n$, where n is the number of processes.

Definition: The update of the vector time follows two rules:

1. Before a send or internal event,

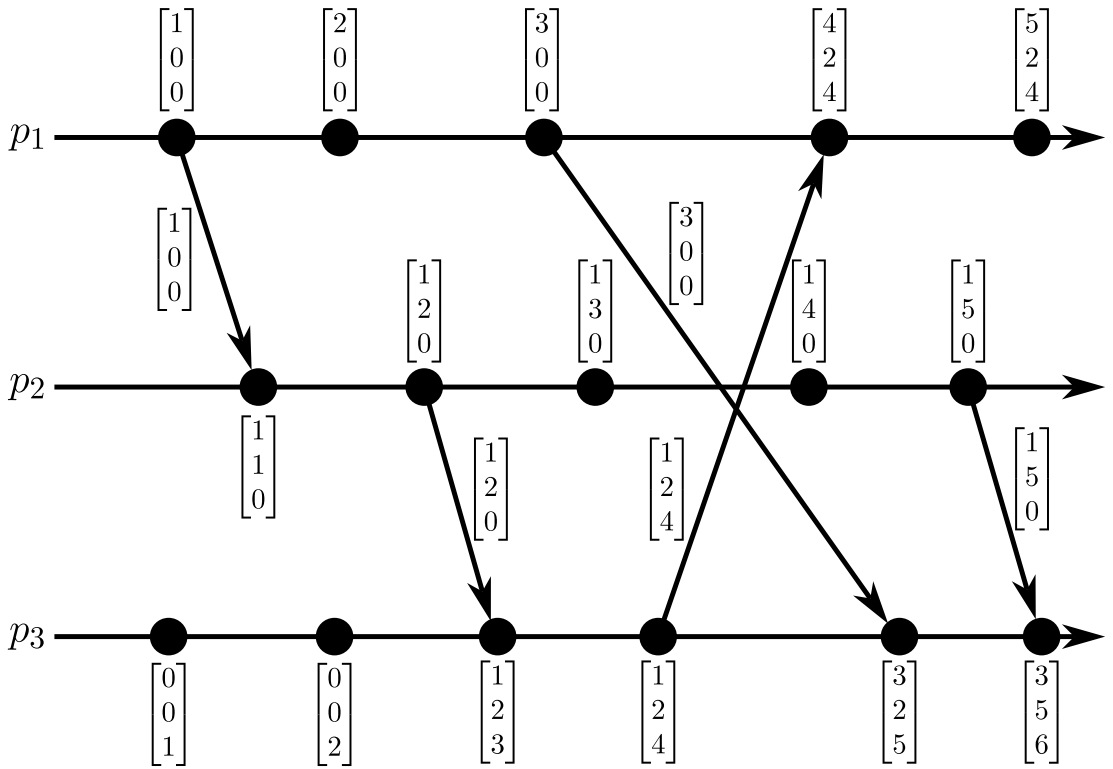
$$vt_i[i] := vt_i[i] + d, \qquad d > 0.$$

   The message to be sent piggybacks the vector vt_i after vt_i is updated.

2. When p_i receives a message with timestamp vt, execute the following actions:
   1. $vt_i[k] := \max(vt_i[k], vt[k])$ for all $k = 1, \cdots, n$;
   2. Execute 1;
   3. Deliver the message.

Example

Assume $d \equiv 1$. All clocks begin with $\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$.



Comparing two vector timestamps

Given $vt_1, vt_2 \in T = \mathbb{N}^n$,

$$vt_1 = vt_2 \iff vt_1[k] = vt_2[k],\ \forall k;$$
$$vt_1 \leq vt_2 \iff vt_1[k] \leq vt_2[k],\ \forall k;$$
$$vt_1 < vt_2 \iff vt_1 \leq vt_2 \text{ and } \exists k \text{ s.t. } vt_1[k] < vt_2[k];$$
$$vt_1 \parallel vt_2 \iff \text{neither } vt_1 < vt_2 \text{ nor } vt_2 < vt_1.$$

Example:

$$\begin{bmatrix}1\\2\\3\end{bmatrix} < \begin{bmatrix}1\\3\\3\end{bmatrix}, \qquad \begin{bmatrix}1\\2\\3\end{bmatrix} \parallel \begin{bmatrix}1\\3\\2\end{bmatrix}.$$

Properties

- **Isomorphism and strong consistency:**

$$e_1 \to e_2 \iff C(e_1) < C(e_2),$$
$$e_1 \parallel e_2 \iff C(e_1) \parallel C(e_2).$$

Suppose vt_i and vt_j are timestamps of e_i^x and e_j^y, respectively. Then

$$e_i^x \to e_j^y \iff vt_i[i] \leq vt_j[i];$$
$$e_i^x \parallel e_j^y \iff vt_i[i] > vt_j[i] \text{ and } vt_i[j] < vt_j[j].$$

- **Event counting:** Suppose $d = 1$ and vt_i is the timestamp of e_i^x. Then
  - $vt_i[i]$ denotes the number of events that have occured at process i including e.
  - $vt_i[j]$ with $j \neq i$ denotes the number of events produced by process j that happen before e.
- **Message complexity:** The overhead is $O(n)$ per message.

Compressed timestamp

General idea: When p_i sends a message to p_j, it only piggybacks those entries of its vector clock that differ since the last message sent to p_j.

Compressed timestamp

General idea: When p_i sends a message to p_j, it only piggybacks those entries of its vector clock that differ since the last message sent to p_j.

Implementation: Each p_i stores n vectors recent τ_{ij}, $j = 1, \cdots, n$, denoting the lastest timestamp when p_i sends a message to p_j.

Compressed timestamp

General idea: When p_i sends a message to p_j, it only piggybacks those entries of its vector clock that differ since the last message sent to p_j.

Implementation: Each p_i stores n vectors recent τ_{ij}, $j = 1, \cdots, n$, denoting the lastest timestamp when p_i sends a message to p_j.

- Initially, $\tau_{ij} = [0, \cdots, 0]$ for all i, j.
- When p_i sends a message to p_j:
  1. Update the current time stamp vt_i.
  2. Compare τ_{ij} with vt_i and find all different entries:

$$\tau_{ij}[k_1] \neq vt_i[k_1], \quad \cdots, \quad \tau_{ij}[k_{n_1}] \neq vt_i[k_{n_1}].$$

  3. The compressed timestamp $\{(k_1, vt_i[k_1]), \cdots, (k_{n_1}, vt_i[k_{n_1}])\}$ is piggybacked to the message.
  4. Set $\tau_{ij} := vt_i$ and send the message.

Compressed timestamp

General idea: When p_i sends a message to p_j, it only piggybacks those entries of its vector clock that differ since the last message sent to p_j.

Implementation: Each p_i stores n vectors recent τ_{ij}, $j = 1, \cdots, n$, denoting the lastest timestamp when p_i sends a message to p_j.

- Initially, $\tau_{ij} = [0, \cdots, 0]$ for all i, j.
- When p_j receives the message from p_i:
  1. Set

$$\begin{aligned} vt_j[k_1] &:= \max(vt_i[k_1], vt_j[k_1]), \\ \cdots \quad & \cdots \quad \cdots, \\ vt_j[k_{n_1}] &:= \max(vt_i[k_{n_1}], vt_j[k_{n_1}]). \end{aligned}$$

  2. Update the timestamp vt_j.
  3. Deliver the message.

Compressed timestamp

General idea: When p_i sends a message to p_j, it only piggybacks those entries of its vector clock that differ since the last message sent to p_j.

Implementation: Each p_i stores n vectors recent τ_{ij}, $j = 1, \cdots, n$, denoting the lastest timestamp when p_i sends a message to p_j.

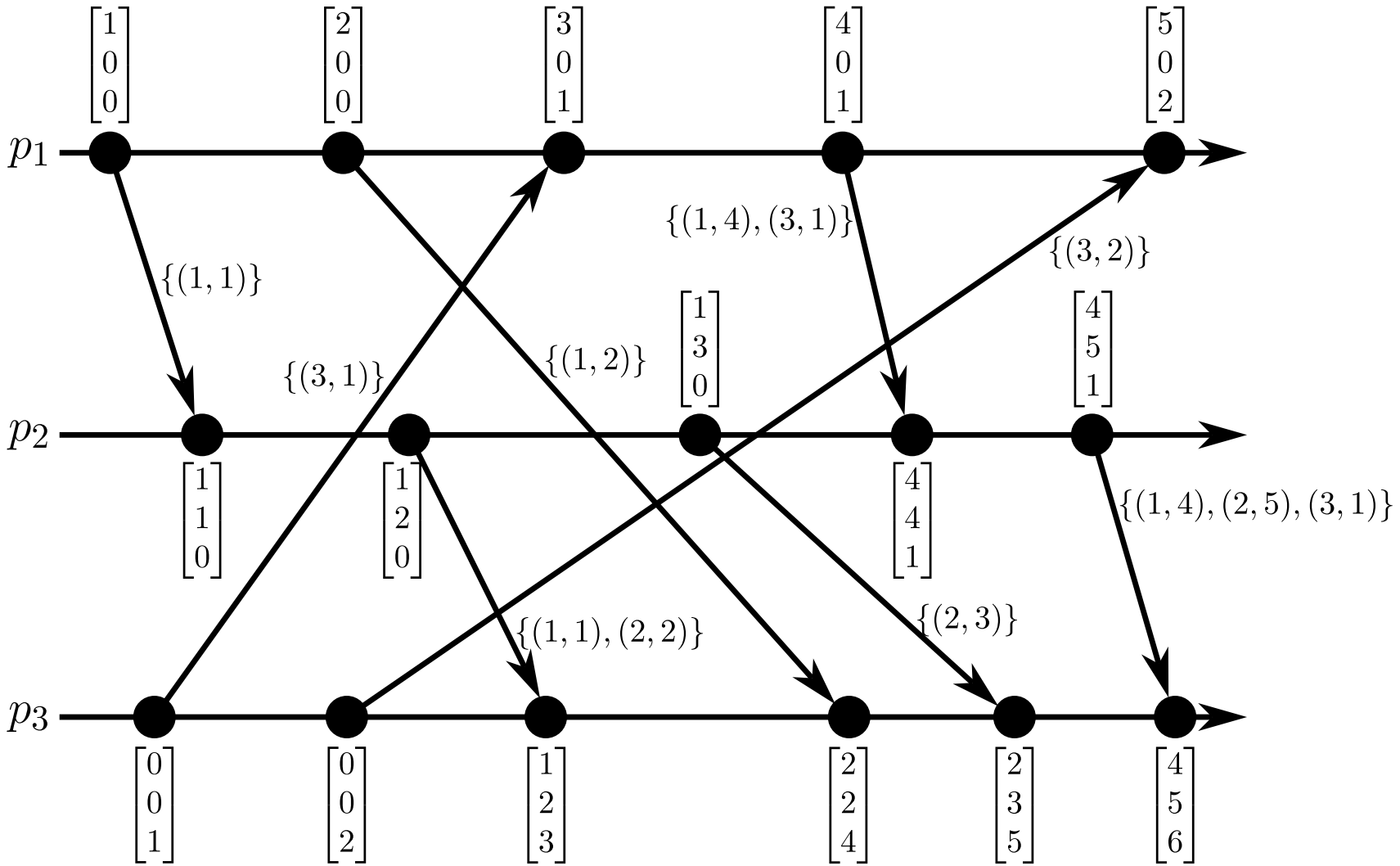
- Initially, $\tau_{ij} = [0, \cdots, 0]$ for all i, j.
- When p_j receives the message from p_i:
  1. Set

$$\begin{aligned} vt_j[k_1] &:= \max(vt_i[k_1], vt_j[k_1]), \\ \cdots \quad & \cdots \quad \cdots, \\ vt_j[k_{n_1}] &:= \max(vt_i[k_{n_1}], vt_j[k_{n_1}]). \end{aligned}$$

  2. Update the timestamp vt_j.
  3. Deliver the message.

Storage overhead: $O(n^2)$

Example


p_1
p_2
p_3
$\begin{bmatrix}1\\0\\0\end{bmatrix}$
$\begin{bmatrix}2\\0\\0\end{bmatrix}$
$\begin{bmatrix}3\\0\\1\end{bmatrix}$
$\begin{bmatrix}4\\0\\1\end{bmatrix}$
$\begin{bmatrix}5\\0\\2\end{bmatrix}$
$\{(1,1)\}$
$\{(1,4),(3,1)\}$
$\{(3,2)\}$
$\{(3,1)\}$
$\{(1,2)\}$
$\begin{bmatrix}1\\3\\0\end{bmatrix}$
$\begin{bmatrix}4\\5\\1\end{bmatrix}$
$\begin{bmatrix}1\\1\\0\end{bmatrix}$
$\begin{bmatrix}1\\2\\0\end{bmatrix}$
$\begin{bmatrix}4\\4\\1\end{bmatrix}$
$\{(1,4),(2,5),(3,1)\}$
$\{(1,1),(2,2)\}$
$\{(2,3)\}$
$\begin{bmatrix}0\\0\\1\end{bmatrix}$
$\begin{bmatrix}0\\0\\2\end{bmatrix}$
$\begin{bmatrix}1\\2\\3\end{bmatrix}$
$\begin{bmatrix}2\\2\\4\end{bmatrix}$
$\begin{bmatrix}2\\3\\5\end{bmatrix}$
$\begin{bmatrix}4\\5\\6\end{bmatrix}$

Singhal-Kshemkalyani's differential technique

Singhal and Kshemkalyani developed an algorithm requiring storage overhead $O(n)$ only:

Singhal-Kshemkalyani's differential technique

Singhal and Kshemkalyani developed an algorithm requiring storage overhead $O(n)$ only:

- Each process p_i maintains only two vectors:
  1. LS_i (Last Sent): $LS_i[j]$ indicates the value of $vt_i[i]$ when process p_i last sent a message to p_j.
  2. LU_i (Last Update): $LU_i[j]$ indicates the value of $vt_i[i]$ when process p_i last updated the entry $vt_i[j]$.

Singhal-Kshemkalyani's differential technique

Singhal and Kshemkalyani developed an algorithm requiring storage overhead $O(n)$ only:

- Each process p_i maintains only two vectors:
  1. LS_i (Last Sent): $LS_i[j]$ indicates the value of $vt_i[i]$ when process p_i last sent a message to p_j.
  2. LU_i (Last Update): $LU_i[j]$ indicates the value of $vt_i[i]$ when process p_i last updated the entry $vt_i[j]$.

Algorithm:

- Initially, set $LS_i = [0, \cdots, 0]$ and $LU_i = [0, \cdots, 0]$ for all i.
- When p_i sends a message to p_j:
  1. Update the current time stamp vt_i.
  2. Piggyback the set of tuples
  $$\{(k, vt_i[k]) \mid LS_i[j] < LU_i[k] \text{ or } k = i\}$$
  to the message.
  3. Set $LS_i[j] := vt_i[i]$ and send the message.

Singhal-Kshemkalyani's differential technique

Singhal and Kshemkalyani developed an algorithm requiring storage overhead $O(n)$ only:

- Each process p_i maintains only two vectors:
  1. LS_i (Last Sent): $LS_i[j]$ indicates the value of $vt_i[i]$ when process p_i last sent a message to p_j.
  2. LU_i (Last Update): $LU_i[j]$ indicates the value of $vt_i[i]$ when process p_i last updated the entry $vt_i[j]$.

Algorithm:

- Initially, set $LS_i = [0, \cdots, 0]$ and $LU_i = [0, \cdots, 0]$ for all i.
- When p_j receives the message from p_i:
  1. For each tuple $(k, vt_i[k])$, if $vt_i[k] > vt_j[k]$, set

$$vt_j[k] := vt_i[k] \text{ and } LU_j[k] := vt_j[j] + 1.$$

  2. Update the timestamp vt_j.
  3. Deliver the message.

1 Models of distributed systems

2 Physical and logical times

3 Summary

Summary

Summary:

- Causal precedence relations and models of communication works in message-passing distributed systems.
- Physical clocks and the network time protocal.
- Logical times (scalar and vector) and implementations.